

Annex XI - WR Pointer & Augmented Overlay

Device-Adaptive Invocation and Presentation Layer for Governed Hybrid AI

Version 1.0 — 19 July 2026 Part of the Optirando™ Canon Status: Defensive Publication / Prior-Art Disclosure

XI.0 Status of this Annex

This annex is published as a defensive disclosure. It describes the WR Pointer™ and the Augmented Overlay as designed components of the Optirando™ platform. Nothing in this annex asserts that any described protection is absolute; the platform is under active development, and all security statements are to be read under the Honest Boundary discipline established in Annex X: the system states what it enforces, what it observes, and what it cannot yet guarantee.

Amendment discipline (normative). This annex is written to be extended and adapted at any time. Amendments are additive: new sections, subsections, embodiments, device classes, action categories, and interaction patterns may be added in later versions, but existing normative invariants (§XI.3) may only be strengthened, never weakened or removed. The Version History (§XI.21) records every change. Silent replacement of any section is prohibited; superseded text is marked as superseded and retained by reference. Section numbers, once assigned, are never reused for different content.

Relationship to the Canon. This annex deliberately does not restate mechanisms specified elsewhere. It references:

- Annex III (PoAC™) for capability acquisition, provenance, and admission discipline;
- Annex VII (BEAP™ handshake architecture) for handshake profiles, capability declaration, Merkle manifests, publisher attestation tiers P0–P2, and the grant model;
- Annex VIII (hybrid compute architecture) for the Augmented Overlay's original introduction, Frames, strict rendering passivity, Mini-Apps built from pre-admitted interaction primitives, Editable Regions, and WR Anchor export;
- Annex IX (initiation layer and WR TED) for consent classes C0–C4, the Intent Hash / WYSIWYE discipline, the execution lifecycle Validate → Execute → Witness → Seal → Anchor, verification tiers L/C/X, and the AI-as-Router characterization;
- Annex X (WR Guard™ and capability-governed information flow) for boundary governance, the three governance classes (Public Handshake, Internal Handshake, External Service Admission), Invocation Assurance Classes, the ten graduated egress measures, Boundary Event Records, and enforcement modes.

Where this annex says "by reference," the referenced annex is normative for that mechanism.

XI.1 Purpose and Scope

Modern AI assistance is delivered through chat windows and detached copilots. The user's actual working surface — the screen content in front of them, the document under the cursor, the machine in front of the camera — is disconnected from the assistance layer, and the assistance layer is

disconnected from any verifiable authorization or evidence mechanism. Where "AI overlays" exist today, they typically combine three dangerous properties: they read arbitrary screen content, they can act on it, and they carry no cryptographic governance of what may enter or leave the trust boundary.

The WR Pointer and the Augmented Overlay together form the interaction layer of the Optirando platform that closes this gap:

- The **WR Pointer** is the device-adaptive **invocation surface**: the single place where the user designates content or objects, bounds context, selects actions, and triggers governed execution.
- The **Augmented Overlay** is the **presentation surface**: the single place where the system displays status, offered actions, contextual help, and context-related information.

Both operate strictly inside the governance model of the Canon: AI prepares, the human authorizes, the system records cryptographic proof. This annex specifies the invariants, artifact classes, declaration requirements, and device-class embodiments of this interaction layer, and it defines how the layer remains extensible without weakening its guarantees.

Out of scope: the internal implementation of any particular model runtime, rendering engine, or operating-system integration. These are embodiment details; the annex constrains their behavior, not their construction.

XI.2 Definitions

WR Pointer™ — the invocation surface of the WR system. A capability architecture with device-adaptive embodiments (§XI.4). Colloquially "the pointer."

Augmented Overlay — the presentation surface of the WR system, first introduced in Annex VIII as cursor-/UI-bound real-time support on composed content, generalized here to all pointer contexts.

Pointer Action — a discrete, declared unit of functionality invocable through the pointer, defined by a Pointer Action Manifest (§XI.6).

Pointer Action Manifest (PAM) — the signed declarative artifact defining a pointer action: identity, origin, trigger context, inputs, permitted context sources, capability and consent requirements, validation, Finalizer reference, and evidence requirements.

Origin Class — the signed provenance category of a pointer action or overlay content item: user-created, local WRC, organization policy, verified publisher (handshake-declared), or AI suggestion (§XI.3, I4).

Trigger Context — the situation class in which an action is available: global, or context-scoped (e.g. WR Code® scan, BEAP™ message, infographic Frame, document selection, camera designation).

Region Marker — the pointer's explicit context-bounding tool: a user-drawn or user-confirmed selection of a screen region, content element, or camera frame that defines the exact context boundary of an action (§XI.8).

Real-World Pointer — the mobile camera embodiment of the pointer: the camera used as a pointing instrument toward physical objects and scenes (§XI.4.3).

Action Registry — the local, host-maintained set of admitted Pointer Action Manifests, filterable by trigger context and origin class (§XI.7).

Terms not defined here (PoAE™, PoAC™, WR Graph™, WR Guard™, WR Expert™, WR TED, WCR, Finalizer, Intent Hash, consent classes C0–C4, Invocation Assurance Class, Boundary Event Record) carry their Canon definitions by reference.

XI.3 Core Invariants (Normative)

The following invariants define the WR Pointer and Augmented Overlay. They hold across all device classes, all embodiments, and all future extensions of this annex. Amendments may add invariants or strengthen these; they may not weaken or remove them.

I1 — The pointer is a trigger, not authority. The pointer never possesses or confers authority of its own. It is the execution trigger for authority that is established elsewhere: in signed manifests, capability grants, policy, and consent. Echoing the canonical formulation of Annex IX for entry codes: *the pointer is an invocation instrument — not authority*.

I2 — Duality of channels: the pointer invokes, the overlay presents. The WR Pointer is the sole invocation channel of the interaction layer; the Augmented Overlay is the sole presentation channel. The overlay displays status, offers, help, and information. The overlay **offers, never acts**: nothing displayed in the overlay carries execution authority until the user invokes it through the pointer under the applicable consent class. The offer-never-action discipline of Annex IX applies to the overlay verbatim. An overlay without an execution path is structurally devalued as an attack surface: fake dialogs cannot execute because no overlay element can.

I3 — Content is data, never instruction. Visible content — screen content, page content, document text, camera imagery, scanned codes beyond their declared identifier role — can never create, propose, parametrize, or alter the semantics of a pointer action. Pointer actions originate exclusively from the four authoritative sources of I4 (excluding raw AI output, which may only rank and select per I6). Selected or captured content is payload for a chosen action, never that action's definition. This extends the instruction-incapable-extraction principle of Annex IX (Assisted Code Capture) to the entire interaction layer.

I4 — Signed origin classes on every action. Every pointer action and every overlay content item carries a verifiable origin class: **(a)** user-created, **(b)** local WRC-derived, **(c)** organization-declared (signed policy), **(d)** verified-publisher-declared (handshake manifest), or **(e)** AI suggestion (a ranking over items of classes a–d, never an origin of new actions). The origin class is displayed unmodifiably wherever the action or content appears, with consistent visual encoding across menu and overlay (§XI.11.3). Local and handshake-associated actions appear in the same menu, distinguished by this encoding — there is no separate "handshake mode"; the trust distinction lives per action, not per mode.

I5 — No hidden path. Everything that crosses the trust boundary in any direction — context leaving the device, publisher content entering, model invocations, adapter deployment, help assets, status telemetry — must be cryptographically and explicitly permitted under the governance classes

of Annex X (Public Handshake, Internal Handshake, External Service Admission) and is logged as Boundary Event Records. There is no exempted flow class for the interaction layer. This includes help texts, thumbnails, previews, and any other "convenience" content.

I6 — AI as router. AI participation in the pointer is confined to the router role established in Annex IX: selecting among pre-admitted actions, ranking offers, binding parameters within declared constraints, and choosing declared branches. AI never synthesizes new actions, new execution paths, or new effect classes at runtime.

I7 — Effect-boundary determinism and the mis-specification rule. Determinism is achieved at the effect boundary, not in the reasoning. Every externally consequential effect passes through deterministic validation against the action's schema, the applicable policy, and the WR TED lifecycle (Validate → Execute → Witness → Seal → Anchor, by reference to Annex IX). From this follows a normative admission rule:

A publisher action whose safety depends on the quality of AI reasoning rather than on deterministic validation at the effect boundary is mis-specified and is not admissible.

I8 — Explicit context bounding. Context acquisition is always explicitly bounded — by the Region Marker, by a designated element, or by a confirmed camera frame — and never implicitly "the whole screen" or "the whole scene." The bounded region is part of the Intent Hash: the user authorizes exactly the bounded context they see (WYSIWYE). Context acquisition destined for any external recipient is an information flow under Annex X and passes through WR Guard.

I9 — Uniform architecture across device classes. The action registry, manifest format, origin classes, consent mapping, evidence chain, and all invariants I1–I8 are identical across device classes. Only the interaction form varies (§XI.4). No device class receives a weaker governance regime because of its form factor.

XI.4 Device-Class Model (Normative)

XI.4.1 Principle

The WR Pointer is a **capability architecture, not a widget**. It manifests in different **embodiments** per device class, and — this is essential to the design — the pointer may carry **different functions per device class**, because different devices afford fundamentally different pointing relationships to the world:

- On the **desktop**, the pointer points at *rendered digital content* with pixel precision.
- On the **smartphone**, the pointer additionally points at *the physical world* through the camera — and separately provides an on-screen equivalent of the desktop pointer in menu form.
- On the **tablet**, both relationships combine, augmented by stylus precision.

A device class therefore defines: (a) which embodiments exist, (b) which trigger contexts are natively available, (c) which capture instruments the pointer commands, and (d) which interaction patterns apply. It never redefines governance (I9).

XI.4.2 Desktop Embodiment — the AI Cursor Pointer

On desktop, the pointer is an **augmented cursor**: closely bound to the object currently hovered or selected, extended with:

- the Region Marker for bounding arbitrary screen areas (e.g. for screenshots and screen-region context, §XI.8);
- an action menu presenting the context-filtered Action Registry, with origin-class encoding per I4;
- inline invocation of actions on selections (text, images, application content, Frames, Editable Regions per Annex VIII);
- initiation and control of WRC recording (§XI.13).

The desktop embodiment is the primary **composition and deep-work surface**: full WR Graph context, the strongest local models, multi-application workflows, and rich Overlay presentation space.

XI.4.3 Mobile Embodiments

The smartphone carries **two distinct pointer embodiments**, both first-class:

(a) The Real-World Pointer (camera embodiment). The camera acts as the pointing instrument toward the physical world — like a finger with which the user points at something. Functions include:

- **designation**: pointing at an object, machine, component, document, or scene to establish it as the subject of an action;
- **descriptive grounding**: the pointed-at imagery gives AI assistance a precise visual referent, enabling substantially better description, identification, and support for images and physical objects than text alone;
- **WR Code® scanning** as the governed entry path (identifier and invitation — not authority; Annex IX by reference);
- **evidence capture** with explicit frame confirmation (the confirmed frame is the bounded context, I8);
- physical-world entry into field-service and DeepFix flows (Annex IV by reference).

The Real-World Pointer obeys I8 strictly: only the confirmed frame or designated region enters any action's context; continuous ambient capture is not a pointer function.

(b) The On-Screen Pointer (menu embodiment). The smartphone equivalent of the desktop cursor: a floating pointer control and a **swipe menu** that lists the available actions — the context-filtered view of the Action Registry. Properties:

- the swipe menu lists all currently available actions, with origin-class encoding;
- users can **add actions** to the menu — both **global** actions and **context-scoped** actions bound to specific trigger contexts (WR Code scan, BEAP message, Frame, document, camera designation, and future context classes); adding an action is a PoAC admission event (§XI.7);
- selection, long-press, and radial patterns per §XI.17; consequential actions never sit in the first interaction layer;
- acts as the approval surface for cross-device consent (§XI.14).

(c) Role profile. The smartphone is primarily the **capture and consent device**: designating, scanning, capturing evidence, issuing short instructions, approving. Its interaction patterns are designed for its form factor, not as a miniature desktop cursor.

XI.4.4 Tablet Embodiment

The tablet combines both relationships: on-screen pointer with stylus-aware precision (stylus hover and press as first-class pointer states), plus the camera embodiment. Stylus-designated regions are Region Marker inputs.

XI.4.5 Invocation Modalities (Normative)

Pointing and interacting are decoupled. On every device class, the user can **point at something — on the desktop screen or in the real world — and then interact with AI about the designated subject** through any of the following modalities:

1. **Speech** — a spoken instruction concerning the designated content or object;
2. **Text** — a typed instruction or query;
3. **Handshake-based AI automation** — invoking a publisher-declared action available for the designated subject (§XI.5, §XI.12);
4. **Self-predefined AI automation** — invoking a user- or organization-defined action from the Action Registry, including WRC-derived actions (§XI.7, §XI.13).

Modalities are orthogonal to embodiments: the same designated subject (a hovered element, a marked region, a confirmed camera frame) can be addressed through any modality the device supports. Governance is modality-independent:

- Speech and text are **user-instruction channels** carrying origin class (a). They express intent; they do not create actions. The instruction is mapped, under I6, onto admitted actions of the registry (or answered as pure local/routed reasoning without consequence); free-form instructions never synthesize new execution paths.
- Only the user's own confirmed input constitutes an instruction. Ambient audio and speech contained in captured content are content, not instruction (I3).
- Whichever modality initiates an action, the action's declared consent class, bounded context, routing constraints, and evidence requirements apply unchanged (I8, §XI.10).

XI.4.6 Future Device Classes (Extension Point)

New device classes — XR/spatial displays, vehicles, wearables, dedicated field hardware — are added to this annex additively as new subsections of §XI.4, each specifying embodiments, trigger contexts, capture instruments, and interaction patterns, under unmodified invariants I1–I9.

XI.5 Operating Contexts: Local and Handshake-Associated Actions

The pointer exposes **one unified surface**. The distinction the user experiences is per action, via origin class and color encoding (I4), not via modes.

Local actions operate without any publisher handshake. They may use local files, selections, images, screenshots (Region-Marker-bounded), application content, the WR Graph (resident access for local models per Annex X; capability-gated for every other requester class), and user-approved system capabilities. Reasoning may be routed to a fully local model, a user-selected external model, an enterprise model, deterministic tools, or combinations — under §XI.9. The user and, where

applicable, signed organization policy control which data may leave the device and which execution environment is used; WR Guard sits on every external path (I5, I8).

Handshake-associated actions become available after a valid Public Handshake (Annex VII). The publisher declares the action's semantics, schemas, capability requirements, consent class, and permitted consequences in its handshake repository (§XI.12). The publisher binds the **effect boundary**; the reasoning layer remains user- or policy-selected within declared assurance requirements (§XI.9). Risk classes are handled exactly as everywhere in the WR system: the applicable consent class, assurance class, and evidence tier follow from the declared risk, and the WR principle for AI-assisted automation applies without exception — declared in advance, consent-gated, evidence-producing, no hidden path.

Both kinds of actions coexist in the same menu and the same overlay, always distinguishable, never interchangeable in provenance.

XI.6 The Pointer Action Manifest (PAM)

XI.6.1 Rationale

WRC recordings are the execution substrate of pointer actions, but execution bodies alone cannot carry declaration, discovery, signing, versioning, and revocation. The PAM is therefore introduced as a distinct declarative artifact class: the manifest is the governed shell; the WRC body (or deterministic tool chain) is what it references.

XI.6.2 Manifest Contents (Normative)

A PAM declares at minimum:

1. **Identity** — stable action identifier, semantic version, author identity, signature;
2. **Origin class** (I4) and, for publisher actions, the handshake and attestation tier (P0–P2, Annex VII);
3. **Trigger context(s)** — global and/or specific context classes (§XI.7);
4. **Inputs** — typed input schema;
5. **Permitted context sources** — an explicit allowlist (selection, region, frame, named graph capabilities, declared assets); anything not listed is not acquirable for this action;
6. **Capability requirements** — the capability tokens the action requires;
7. **Consent class** — the minimum class C0–C4, with the accepted consent mechanisms (click, biometric such as fingerprint, 2FA hardware key, passkey, hardware attestation) mapped per §XI.10;
8. **Assurance requirements** — including, where declared, a minimum Invocation Assurance Class and a minimum **reasoning-provenance class** (§XI.9.2);
9. **Mandatory transformations** — WR Guard measures that are preconditions of invocation (Annex X);
10. **Validation rules** — the deterministic schema and policy checks at the effect boundary;
11. **Finalizer reference** — for consequential actions, the Finalizer and WR TED envelope by reference;
12. **Evidence requirements** — receipt and PoAE™ record contents, verification tier (L/C/X);
13. **Execution body reference** — the WRC recording or declared deterministic chain;

14. **Optional adapter reference** — publisher LoRA adapter(s) per §XI.9.3;

15. **Revocation conditions** and update path.

XI.6.3 Lifecycle

PAMs are acquired, admitted, mutated, and revoked exclusively under PoAC (Annex III): evidence-backed proposals, review, signed admission — *propose, never silently apply*. Updates follow the versioned-manifest discipline of Annex VIII: never silent replacement; the user or organization admits the new version. Revocation removes availability immediately across all device classes and is itself an evidenced event.

XI.7 Action Registry and Context-Scoped Availability

The host maintains the **Action Registry**: the set of admitted PAMs. Every menu the pointer shows — the desktop action menu, the mobile swipe menu, radial layers, overlay offers — is a **filtered view of this one registry**, filtered by the current trigger context and sorted under §XI.11.2.

Context scoping (normative). Actions are available either globally or bound to declared trigger contexts (WR Code scan, BEAP message, Frame, document selection, camera designation, and additively extendable context classes). Users and organizations can add actions **into a specific context**, not only globally. Adding an action to a context — whether newly created, WRC-derived, org-shared, or publisher-offered — is a PoAC admission event with the corresponding origin class, review step, and signature. "Activating" an action is never a governance-free toggle.

Sharing. Because availability is manifest admission, sharing actions between users, devices, and organizations reduces to transporting signed PAMs and admitting them — cryptographically provable, analogous to WR Anchor export (Annex VIII). Portability is thereby a property of the artifact class, not a separate mechanism.

XI.8 Region Marker and Context Acquisition

The Region Marker is integrated into the pointer on all device classes: rectangular/freeform screen regions and element selection on desktop and tablet; element selection and confirmed camera frames on mobile.

The Region Marker is a **governance instrument**, not merely a screenshot tool:

1. **The region is the context boundary.** Only content inside the marked region (or designated element / confirmed frame) enters the action's context. Content outside it does not leave the device for this action (I8).
2. **The region is part of the Intent Hash.** The user authorizes exactly the marked excerpt they see (WYSIWYE, Annex IX by reference). A change of region invalidates the pending consent.
3. **WR Guard sits on the acquisition path — architecturally, not optionally.** Before any bounded context is routed to an external recipient, WR Guard classifies it against the applicable WR Expert artifacts and applies the policy-selected measures of Annex X (warn, block-until-release, mask, pseudonymize, obfuscate, distribute, abstract, local-only,

- recipient-bound encrypt, refuse). The applicable enforcement mode (advisory/enforced) follows the finding class and policy.
4. **Egress preview.** Where the flow permits, the pointer surfaces WR Guard's classification and intended transformations **before** the boundary is crossed, in the overlay, so the user sees the redaction, masking, or distribution plan ahead of disclosure.
 5. **Every acquisition that crosses a boundary emits a Boundary Event Record** (Annex X), at the evidence tier the flow's assurance class requires.
-

XI.9 Model Routing and Publisher LoRA Adapters

XI.9.1 Routing

A pointer action separates intelligence from effect. Reasoning for an action may be performed by a fully local model, a private enterprise model, an approved external provider (under External Service Admission, Annex X), a publisher-provided model, or hybrid arrangements — including local context preparation with external reasoning, external reasoning with local execution, distribution across multiple models with partitioned context (Distribution Obfuscation, Annex VIII/X by reference), and local models selecting an external specialist. Routing is decided by the user and, where present, signed organization policy. Routing never changes the effect boundary (I7).

XI.9.2 Publisher Constraints on Reasoning (Bounded)

A publisher may not bind an action to a specific model product. A publisher **may** declare properties: a minimum reasoning-provenance class (e.g. "attested local model required," where the action's context includes publisher-confidential material) and a minimum Invocation Assurance Class. These are property constraints, verifiable at invocation, compatible with user sovereignty. Any attempt to make action safety depend on a particular model's behavior triggers the mis-specification rule (I7) and blocks admission.

XI.9.3 Publisher LoRA Adapters

A publisher may ship LoRA adapters (or comparable lightweight model augmentations) as part of the handshake package, to improve reasoning quality on its declared actions (domain vocabulary, product structures, form semantics). Normative constraints:

1. An adapter is a **capability acquisition under PoAC**: declared in the handshake's Merkle manifest, signed, versioned, admitted by proposal, revocable. Silent adapter updates are prohibited.
 2. **Scope binding**: a publisher adapter is active only for that publisher's actions and only over the context sources those actions declare. It receives no access to local Graph context or device content beyond the declared allowlist. An adapter is never a side channel.
 3. **Safety independence**: the effect boundary validates identically with or without the adapter (I7). Adapters improve quality; they never carry safety.
 4. Adapter activation and deactivation are evidenced events.
-

XI.10 Consent, Authorization, and Evidence

Every consequential pointer action follows the WR TED lifecycle by reference (Annex IX): **Validate** → **Execute** → **Witness** → **Seal** → **Anchor**, with verification tiers Local / Central / External (Solana anchoring) as declared.

Consent classes and mechanisms. PAMs map to consent classes C0–C4. Consequential actions can be secured with graduated mechanisms — click confirmation, biometric confirmation (e.g. fingerprint), 2FA hardware key, passkey-protected approval, hardware attestation — as established in Annex IX. The mapping of mechanisms to classes is policy-configurable and is designed to be **adapted to the assurance classes** of Annex X in later versions of this annex (extension point: §XI.20); the invariant is only that a class's minimum can be raised by policy, never lowered.

Intent Hash contents. For pointer actions, the Intent Hash covers at minimum: the action identity and version, the bounded context (region/frame digest), the bound parameters, the origin class, the routing class of the reasoning that produced the parameter binding, and the declared consequences presented to the user. The user authorizes exactly this (WYSIWYE); any deviation invalidates consent.

Evidence. Execution produces the declared receipts and PoAE records; boundary crossings produce Boundary Event Records; capability acquisitions (actions, adapters) produce PoAC provenance. Optionally, a pointer session can emit a sealed **session receipt** aggregating its evidenced events — of direct value in field-service contexts (Annex IV).

Offline discipline per §XI.15: consent is never pre-staged against a context that may change before execution.

XI.11 The Augmented Overlay

XI.11.1 Role

The Augmented Overlay is the presentation channel of the interaction layer (I2). It harmonizes with the pointer as its display counterpart and shows:

- **status messages** — including the live lifecycle states of governed execution: the user sees, in the overlay, that validation passed, that execution is witnessed, that the seal is set, that anchoring is confirmed. The overlay is the *visible face of the evidence chain*, turning the cryptographic machinery into an experienced property rather than a hidden one;
- **additional and offered actions** — context-relevant entries from the Action Registry, ranked per §XI.11.2, always as offers (I2);
- **contextual help and information** — assistance bound to the content or object under the pointer.

XI.11.2 Suggestion Ranking

Overlay offers and menu ordering may be ranked by AI — bounded by I6 (selection among admitted actions only). Ranking may reuse the Bayesian priority machinery with user-feedback evidence specified in Annex VIII, and may draw on the WR Graph for personalization and recall. Ranking never introduces, hides the provenance of, or auto-executes an action.

XI.11.3 Provenance Consistency

The origin-class visual encoding (I4) is identical across the pointer menus and the overlay. A publisher-supplied help panel carries the same publisher badge as the publisher's declared action; a local AI suggestion carries the same encoding as a local action. Menu and overlay never diverge in provenance presentation.

XI.11.4 Overlay Content Origins (Closed Set)

Overlay content originates from exactly two sources — there is no third path (I5):

1. **Declared handshake assets.** Publisher help content, product information, status texts, and interactive elements are declared asset classes in the handshake's Merkle manifest: versioned, signed, updated only via the versioned update path. They render under the strict passivity discipline of Annex VIII; interactive elements are Mini-Apps composed declaratively from the local library of pre-admitted interaction primitives, with all interaction logic supplied exclusively by the host runtime. No publisher scripts.
2. **Local artifacts.** Local help and information derive from WR Expert artifacts, the WR Graph, and local system state, with local provenance.

XI.11.5 Overlay on Composed Content

On Frames and infographics (Annex VIII), the overlay additionally supports Smart Actions and Editable-Region interaction as already specified there; this annex adds only the uniform origin encoding and the I2 duality on top of the existing mechanics.

XI.12 Handshake Declaration Requirements

For publishers, pointer and overlay functionality is automation like any other in the WR system and is treated with the same strictness:

1. The publisher's handshake repository must declare **every** pointer action, every overlay asset class, every Mini-App composition, every adapter, and — explicitly — **every AI automation offered for the WR Pointer and the Augmented Overlay**, in the capability-declaration and Merkle-manifest structures of Annex VII.
2. Undeclared pointer/overlay functionality does not exist for the client: it cannot be offered, rendered, or invoked.
3. Declarations carry the publisher's attestation tier (P0–P2) and are subject to the grant model of Annex VII; the user's admission of a publisher's pointer actions is a consent event, and risk classes drive the required consent and evidence per §XI.10.
4. Changes flow only through versioned manifest updates — never silent replacement — and revocation is honored immediately (§XI.6.3).

XI.13 WRC Integration

WRC recordings can be initiated and controlled through the WR Pointer on all device classes. The design separates **learning from authorizing**:

1. **Recording** (start, pause, stop, annotate) is a local, non-consequential C0 pointer action. The recording itself acquires no authority.
2. **Admission** of a finished recording as a new pointer action — globally or into a specific trigger context — is the governance moment: a PoAC proposal with review, PAM generation, and signature (§XI.6, §XI.7). Only admission makes the recording invocable.
3. Recordings replayed as actions execute deterministic-at-tool-boundary, adaptive-on-path, per the WCR discipline; where consequential, under the full §XI.10 chain.
4. Organization-shared and publisher-offered WRC-derived actions follow the same path with their respective origin classes.

XI.14 Cross-Device Continuity

Pointer sessions are continuable across devices via the Host, with the session state carried as a WR Graph object. The canonical division of labor:

- **Smartphone**: capture (Real-World Pointer, WR Code scan, evidence), short instructions, queueing work, and **approval**;
- **Desktop**: continuation with the full Graph, stronger local models, rich editing, multi-application composition, and the full overlay surface.

Phone-as-Approver (normative option). A consequential action initiated on the desktop may require its consent step to be completed on the user's enrolled smartphone: the phone displays the Intent Hash content (WYSIWYE) full-screen and captures the consent mechanism (biometric, hardware key, passkey). The phone thereby serves as the possession-factor approval device inside the graded consent mechanics of Annex IX — unifying cross-device continuity and consent hardening in one mechanism. Enrollment of an approver device is itself a capability-governed, evidenced act.

Session hand-over never transports authority implicitly: capabilities and pending consents are re-validated on the receiving device (Locality-Independent Trust, Annex X).

XI.15 Offline Behavior

- **Local actions** remain fully functional offline: local models, admitted WRCs, resident Graph access.
 - **Handshake-associated actions**: PAMs and declared assets are cacheable within their declared validity windows. Consequential actions may be queued offline for deferred finalization **only** where the manifest explicitly permits it; otherwise the pointer refuses cleanly rather than degrading governance.
 - **Consent is never pre-staged** for later execution against potentially changed context; the Intent Hash binds the state at consent time, and deferred execution re-validates before finalizing.
 - All deferred events are evidenced upon reconnection at their declared tiers.
-

XI.16 Threat Model and Honest Boundary

This section names the principal risks and the design's posture toward them, without asserting absolute protection.

Overlay clickjacking / fake dialogs. Structurally neutralized by I2: overlay elements carry no execution path. Residual risk: user confusion between overlay offers and pointer invocation surfaces; mitigated by consistent provenance encoding and full-screen consent steps for consequential actions.

Injection via visible content. Screen and camera content attempting to smuggle instructions is neutralized by I3 and I6: content cannot define or alter actions, and AI cannot synthesize paths. Residual risk: content influencing AI *ranking* of admitted offers; bounded because ranking can neither add actions nor change their declared semantics or consent class.

Pointer chrome spoofing. A page or app imitating the pointer's UI to harvest interactions. Honest Boundary statement: application-level rendering cannot fully exclude imitation on today's operating systems. Posture: Phase-1 deployment on WR-owned surfaces where the host controls the compositor entirely (§XI.18); host-rendered chrome outside page compositors where platform APIs allow; consequential consent always in host-controlled full-screen surfaces with Intent-Hash display; OS-level trusted-UI integration as the declared evolution path — extending, not replacing, platform security mechanisms, in line with the Annex X trajectory.

Exfiltration through context acquisition. The pointer sees what the user sees; governance, not blindness, is the control: I8 bounding, mandatory WR Guard on the acquisition path, Annex X measures, Boundary Event Records for every crossing and every find.

Overlay permission / accessibility-API abuse (mobile). The mobile embodiments request only the permissions their declared functions need; ambient capture is not a pointer function; all accessibility-service usage (Phase 3) is declared, policy-visible, and evidenced.

Consent fatigue. Graduated classes (C0–C4) keep friction proportional to consequence; pre-admitted policy (organizational) can authorize low-risk classes; consequential classes deliberately retain friction. Ranking (§XI.11.2) reduces offer noise.

Accidental activation (mobile). Long-press-gated menus, no consequential actions in first interaction layers, distinct full-screen consent — §XI.17.

Malicious or defective action manifests. PoAC admission with review and signature; mis-specification rule (I7) at admission; revocation with immediate effect; versioned updates only.

XI.17 Interaction Patterns (Informative)

Desktop. Hover-bound augmentation with subtle affordance; click or shortcut opens the context menu (registry view); drag opens the Region Marker; consequential invocations always route through a host-owned confirmation surface displaying the Intent-Hash content.

Mobile, on-screen. Floating control, movable and dismissible; long-press opens the radial or swipe menu — never hover- or content-triggered; swipe menu lists context-filtered actions with an explicit "add action" flow (global or into the current context, each an admission event);

consequential actions require a separate full-screen consent step with biometric/hardware-key capture as declared.

Mobile, real-world. Point camera; the overlay annotates the recognized designation candidate; user confirms the frame or object explicitly (the confirmation is the bounding event); context-scoped actions for camera designation then appear in the swipe menu.

Tablet. Stylus hover as pre-selection, stylus lasso as Region Marker, otherwise combining both mobile and desktop patterns.

Accessibility (design goal, not afterthought). A single, uniform, declaratively described action surface is inherently more tractable for assistive technologies than heterogeneous per-app UIs: every action carries a machine-readable manifest (name, inputs, consequences, consent class), enabling consistent screen-reader announcement, switch access, and voice invocation across all contexts. Accessibility mappings are a first-class part of each embodiment's interaction pattern set.

XI.18 Staged Implementation (Informative)

- **Phase 1 — WR-owned surfaces (no OS integration).** Pointer and overlay on WR Desk's own rendering surfaces: TimesDesk™ Frames, composed infographics, BEAP message views, Editable Regions. Full compositor control; the spoofing problem does not arise; the complete governance chain is exercised end-to-end.
- **Phase 2 — Browser integration.** Extension-based selection, Region Marker, and handshake actions on WR-connected pages; WR Anchor as the natural attachment point.
- **Phase 3 — Cross-application (accessibility services / AX APIs).** App-spanning selection and designation on desktop and Android; the threat-model items of §XI.16 concerning overlays and accessibility permissions become primary.
- **Phase 4 — OS-level integration.** Trusted pointer chrome, capability-token checks below the application layer (extending platform mechanisms such as LSM-based controls rather than replacing them, per the Annex X trajectory).

Each phase is additive; no phase relaxes an invariant.

XI.19 Prior Work and Differentiation (Informative)

Assistive cursors, floating assistant buttons, OS share-sheets, browser copilots, screen-reading AI overlays, and "click-to-ask" tools are established categories. The disclosed combination differs in the governance binding, which to the author's knowledge is not present in these categories:

- signed, versioned, revocable **action manifests** with declared context-source allowlists and consent classes;
- a strict **invocation/presentation duality** in which the presentation layer is structurally incapable of execution;
- **content-is-never-instruction** across screen and camera input, with AI confined to routing over pre-admitted actions;
- the **mis-specification rule** making effect-boundary determinism an admission criterion;
- mandatory **information-flow governance** (WR Guard, Boundary Event Records) on every context acquisition;

- device-adaptive embodiments — including the camera as Real-World Pointer — under one uniform governance architecture;
- cross-device **Phone-as-Approver** consent inside a cryptographic evidence chain (PoAE, WR TED lifecycle, L/C/X verification).

Essential combined disclosure. For the avoidance of doubt, this annex discloses, as a combination: a device-adaptive pointer serving as sole invocation surface over a signed action registry with origin classes and context-scoped availability; an overlay serving as sole presentation surface under offer-never-action; explicit region-/frame-bounded context acquisition hashed into user consent; user-/policy-controlled model routing with publisher-shipped, PoAC-governed LoRA adapters scoped to declared actions; publisher declaration of all pointer/overlay AI automations in handshake repositories; WRC recording triggered by the pointer with admission-gated activation; cross-device continuation with smartphone-based cryptographic approval; and deterministic finalization with graded consent mechanisms and multi-tier verification. Each element and the combination are hereby published as prior art.

XI.20 Extension Points and Amendment Discipline

Declared extension points for future versions (additive only):

1. new device classes and embodiments (§XI.4.6);
2. new trigger-context classes (§XI.7);
3. refined mapping of consent mechanisms to assurance classes (§XI.10);
4. additional overlay content forms within the closed origin set (§XI.11.4);
5. session receipts and their evidence schema (§XI.10);
6. Phase-3/4 platform integration specifics (§XI.18);
7. accessibility mapping specifications (§XI.17);
8. adapter classes beyond LoRA under the same §XI.9.3 constraints.

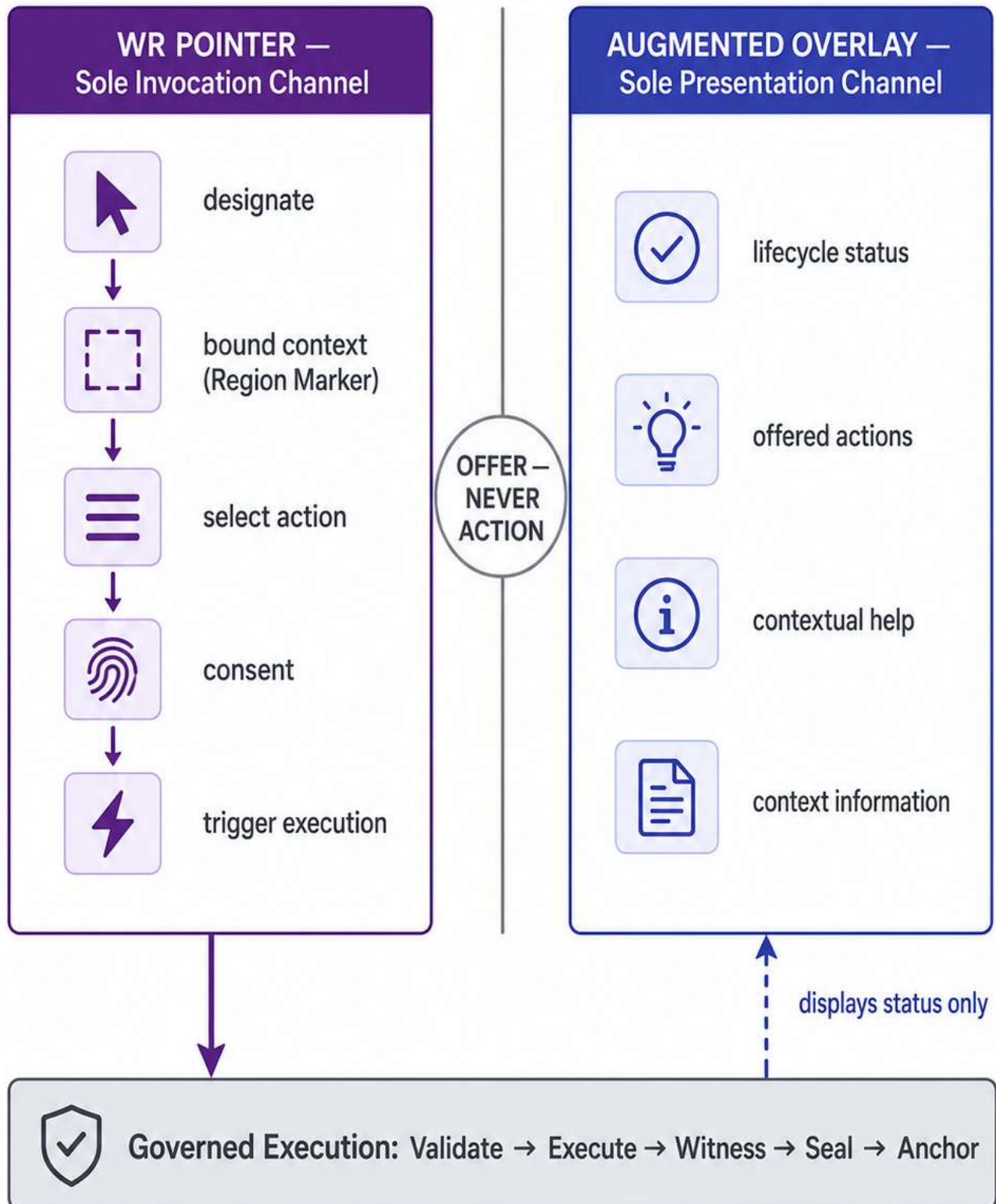
All amendments follow §XI.0: additive, versioned, never weakening I1–I9, recorded in §XI.21.

XI.21 Version History

- **v1.0 — 19 July 2026.** Initial version. Core invariants I1–I9; device-class model with desktop AI cursor, mobile Real-World Pointer and On-Screen Pointer, tablet hybrid; invocation modalities (speech, text, handshake-based and self-predefined AI automation) decoupled from pointing; unified action surface with origin classes; Pointer Action Manifest and Action Registry with context-scoped admission; Region Marker as governance instrument; model routing with publisher LoRA adapters under PoAC; consent/evidence integration by reference to Annex IX/X; Augmented Overlay as sole presentation channel with closed content-origin set; handshake declaration requirements; WRC integration; cross-device continuity with Phone-as-Approver; offline discipline; threat model with Honest Boundary; informative interaction patterns, staged implementation, prior-work differentiation; extension points.

License Notice. This Annex forms an integral part of the Optirando™ specification and is licensed under the same terms as BEAP™, WR Desk™, and PoAE™.

WR Pointer & Augmented Overlay — Two Channels, One Architecture



The overlay carries no execution path — fake dialogs cannot act.

One Governance Architecture — Device-Adaptive Embodiments



End-to-End Governed Actions (Examples)



Only the interaction form varies — never the governance.

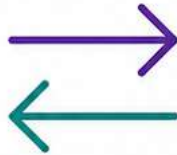
Nine Core Invariants (I1–I9)

I1 Trigger, not authority



the pointer never owns authority.

I2 Pointer invokes, overlay presents



offer-never-action.

I3 Content is data, never instruction



screen and camera content cannot create actions.

I4 Signed origin classes



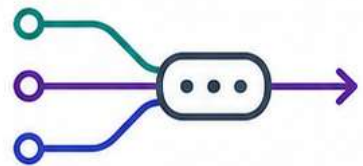
every action shows verified provenance.

I5 No hidden path



every boundary crossing cryptographically permitted.

I6 AI as router



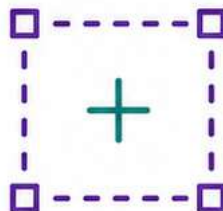
selects among admitted actions, never synthesizes paths.

I7 Deterministic effect boundary



safety lives in validation, not reasoning.

I8 Explicit context bounding



the marked region is what the user authorizes.

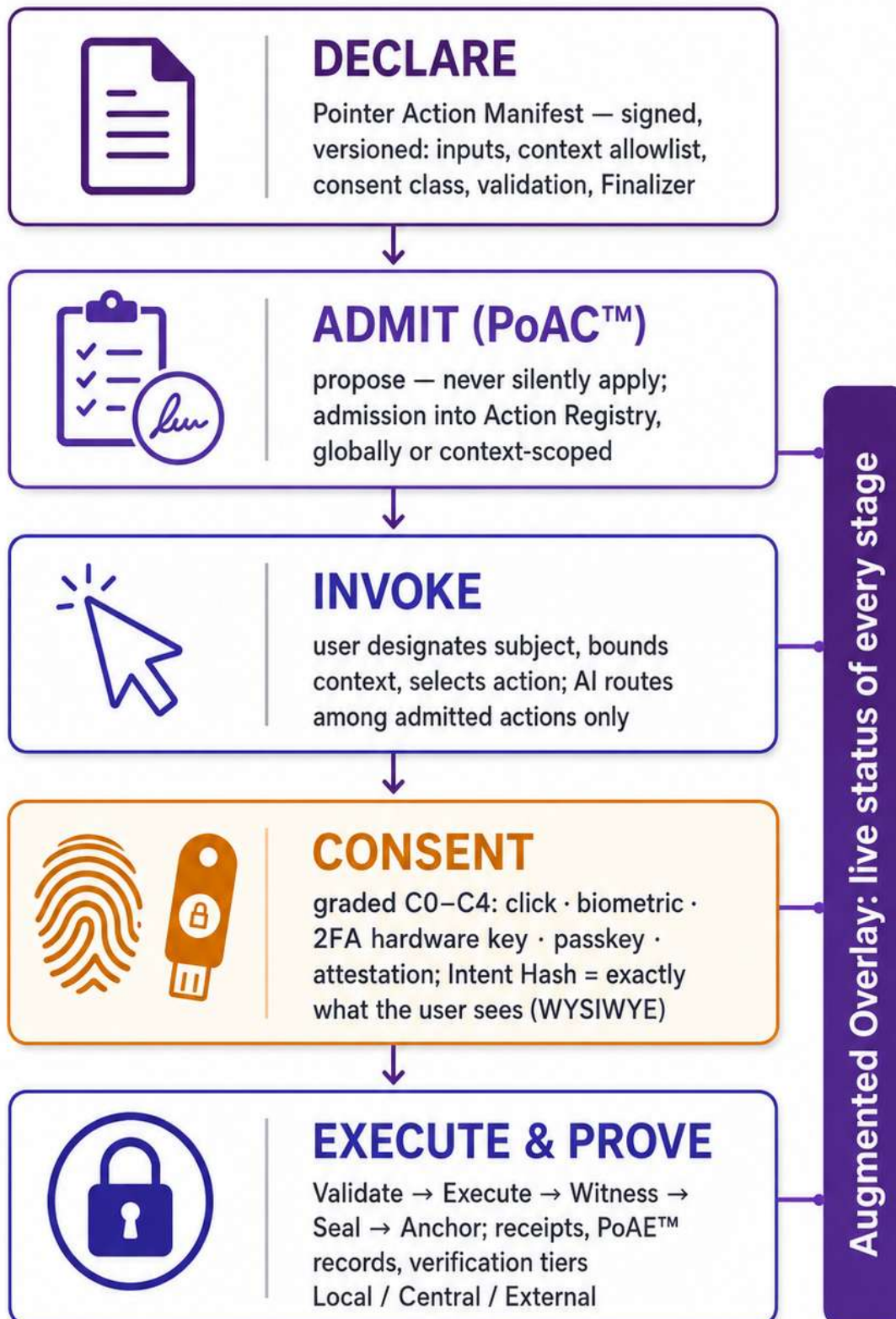
I9 Uniform across devices



no weaker regime for any form factor.

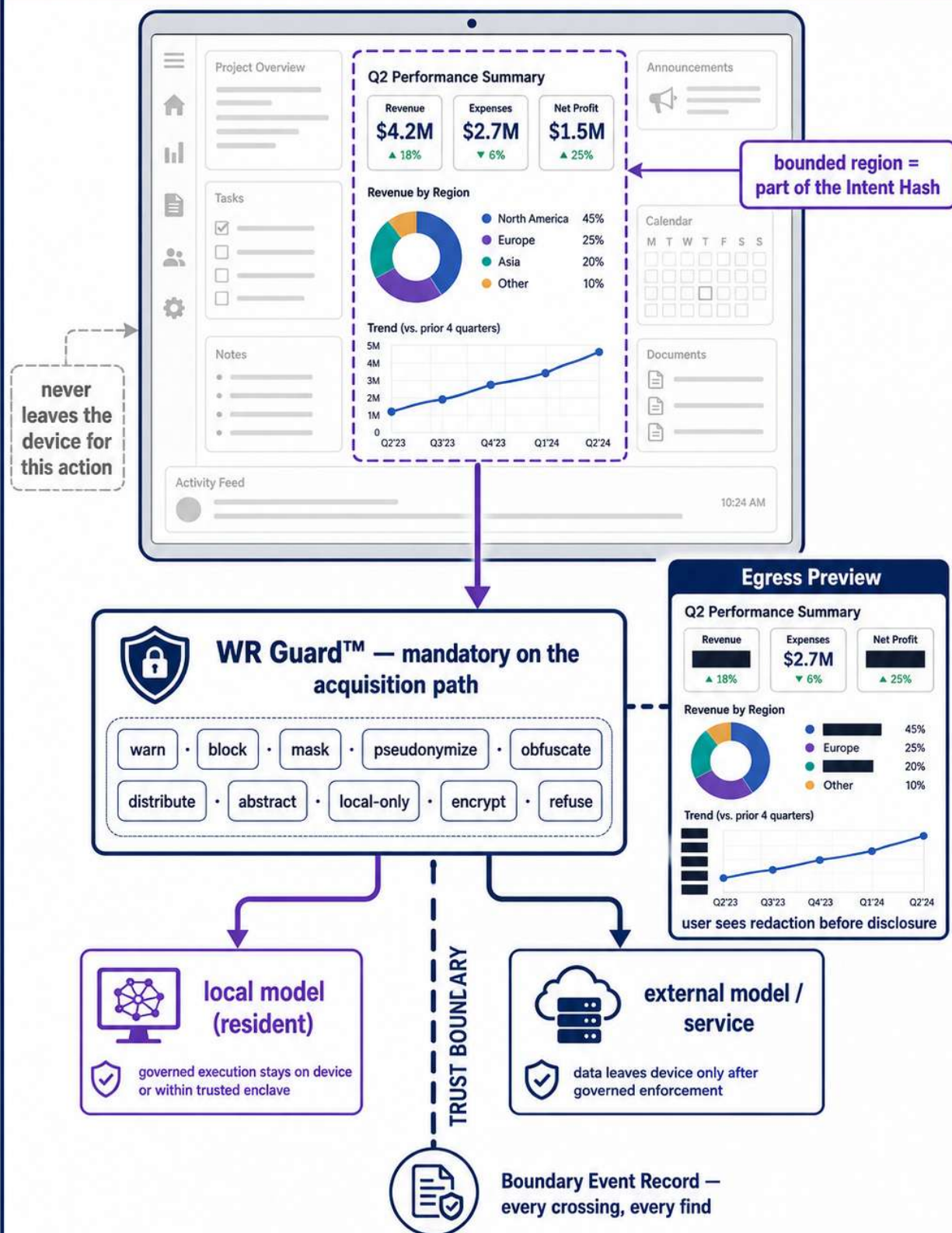
Amendments may strengthen invariants — never weaken them.

From Declaration to Cryptographic Proof – the Pointer Action Lifecycle



Revocation is immediate; updates are versioned — never silent.

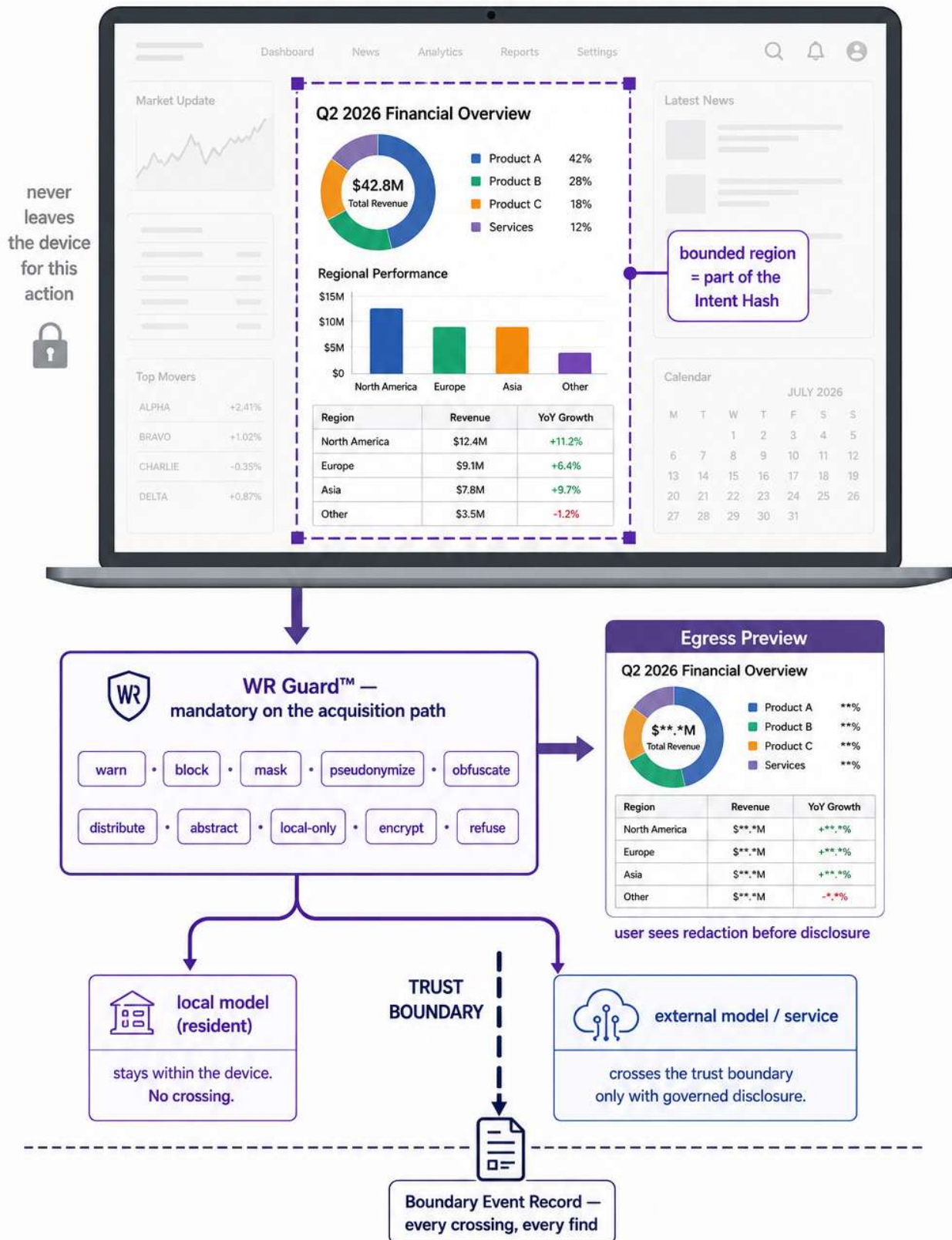
The Region Marker – Context Bounding as a Governance Instrument



Never implicitly the whole screen.

Figure 5 — Region Marker & Governed Context Acquisition

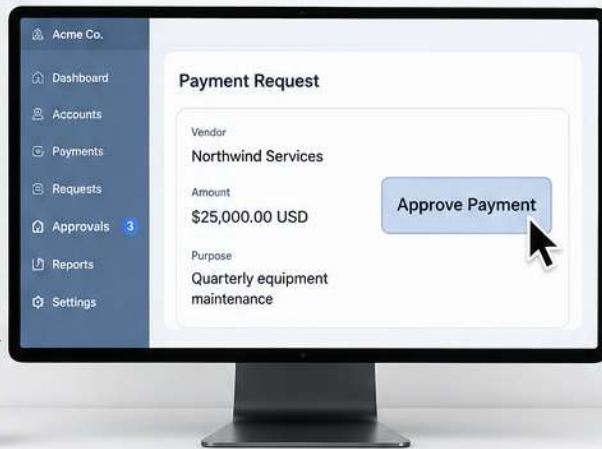
The Region Marker — Context Bounding as a Governance Instrument



Never implicitly the whole screen.

Designate → Interact → Continue

1 POINT



screen or real world — the designated subject

2 INTERACT — four modalities, one governance



Speech



Text



Handshake-based
AI automation



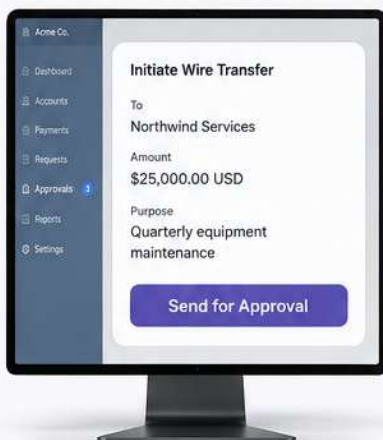
Self-predefined
AI automation



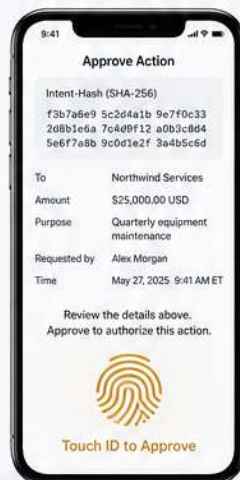
same consent class, same bounded context, same evidence — modality-independent

3 CONTINUE — Phone-as-Approver

Desktop initiates
consequential action



Phone presents full-screen
approval with Intent-Hash



Back to desktop —
finalized with cryptographic proof



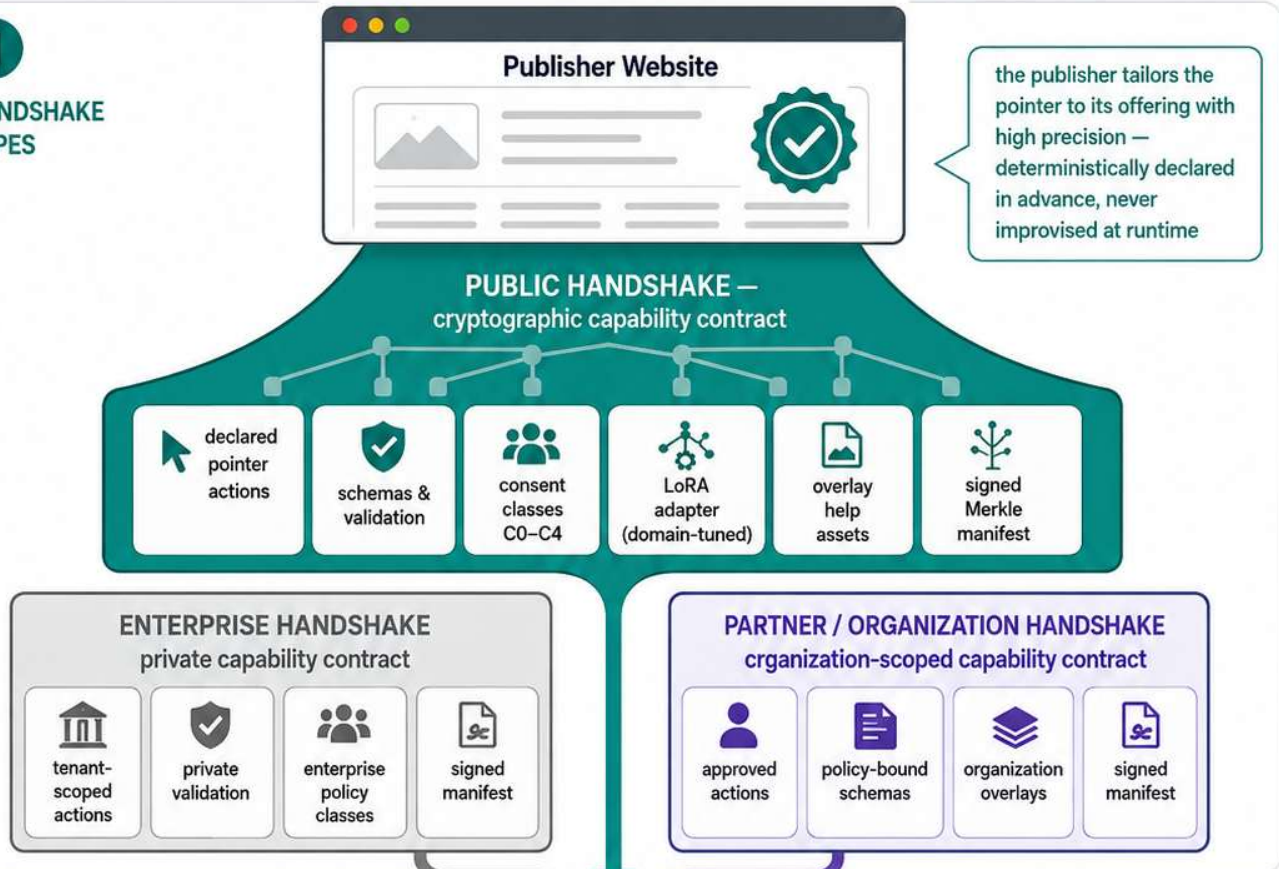
The smartphone is the capture and consent device;
the desktop is the composition surface.

The Pointer Is the Trigger

Public handshake types and local pointer capabilities under one pointer

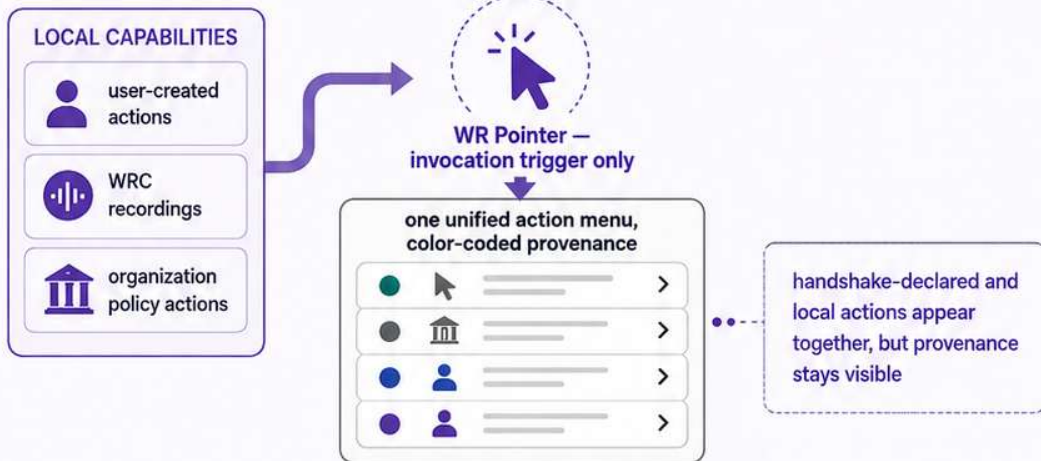
1

HANDSHAKE TYPES



2

POINTER + LOCAL CAPABILITIES



3

PROVENANCE SUMMARY

one pointer, one menu — multiple governed capability origins



Public and private handshake capabilities, plus local pointer capabilities — unified under one pointer with visible provenance.